**Case Study 4:** Investigating Rhion Possession Evidence

**Student Name:** Ibrahim Diseh Garba

**Registration Number:** 2025/FWSD/11521

**Programme:** Fellowship in Web Application Security & Digital Forensics

**Course:** SBT-DF204 — Computer Forensics Case Studies

**Case Title:** Investigating Rhion Possession Evidence

**Instructor:** Aminu Idris, AMCPN

**Date:** 22 October 2026

**1. Executive Summary**

This report presents the findings of an offline digital forensic investigation into a suspected case of illegal possession and distribution of prohibited imagery (rhinoceroses). The investigation analysed a seized USB disk image (RHINOUSB.dd) alongside three authorised network traffic captures (rhino.log, rhino2.log, and rhino3.log).

Across four independent forensic techniques, nine unique rhinoceros images were recovered and validated by SHA-256 hashing, meeting the possession threshold described in the case narrative. Deleted-file carving recovered four, FTP stream reconstruction recovered three (one duplicated across a cracked ZIP archive), and HTTP object extraction recovered two.

The technical evidence establishes a clear timeline of file deletion, concealment, and transfer. However, while the artefacts strongly indicate the actions taken on the host device, the logs alone cannot definitively prove human identity, highlighting the distinction between machine attribution and person-level attribution (Lyle et al., 2022).

Rhino count

| Source | Rhinos |
|---|---|
| Technique 1 — PhotoRec carving | 4 |
| Technique 2 — Steganography | 2 detected (extraction documented as limitation) |
| Technique 3 — FTP + protected archive | 3 |
| Technique 4 — HTTP | 2 |
| Total unique extracted | 9 |

Confidence Level: High (≈ 90%) for the possession threshold being met. Moderate (≈ 60-70%) for personal attribution — the evidence establishes network activity from specific IP addresses, not physical possession by a named person.

**2. Evidence Acquisition and Integrity**

2.1 Evidence Source

| Field | Value |
|---|---|
| USB image | RHINOUSB.dd — 259,506,176 bytes (247.48 MiB) |
| FTP capture | rhino.log — 3,187,907 bytes |
| HTTP capture 1 | rhino2.log — 292,604 bytes |
| HTTP capture 2 | rhino3.log — 226,094 bytes |
| Source | ICDFA SBT-DF204 Case Study 4 lab package (DFRWS2005-RODEO.zip) |
| Acquisition Date | 4 October, 2026 |

2.2 Working Copy Creation

```
cd ~/rhion
mkdir -p working reports screenshots
cp --preserve=timestamps RHINOUSB.dd working/RHINOUSB_working.dd
sha256sum working/RHINOUSB_working.dd | tee reports/working_copy_sha256.txt
# Verify
ORIG=$(awk '/RHINOUSB.dd/ {print $1}' evidence_sha256.txt)
WORK=$(awk '{print $1}' reports/working_copy_sha256.txt)
[ "$ORIG" = "$WORK" ] && echo "HASHES MATCH"
```

2.3 Integrity Verification Output

Evidence SHA-256:

| File | SHA-256 |
|---|---|
| RHINOUSB.dd | ce550424200a997c61b413941c8ef4df9619a2f96579674952294a176a32be65 |
| rhino.log | 64e6d55b76660eb3aaa41572c1d04e4452510a343bbf42e844424827dedfddb2 |
| rhino2.log | 41939d5de0556b70279056572dee44b6fd84cd05b0788cfd0b6f52f37b161dde |
| rhino3.log | 7b0304f5e88a30c305a99b5a1e2977bced5b9c94da6f458f887bb39966bbfc46 |

Evidence MD5:

| File | MD5 |
|---|---|
| RHINOUSB.dd | 80348c58eec4c328ef1f7709adc56a54 |
| rhino.log | c0d0093eb1664cd7b73f3a5225ae3f30 |
| rhino2.log | cd21eaf4acfb50f71ffff857d7968341 |
| rhino3.log | 7e29f9d67346df25faaf18efcd95fc30 |

3. Lab Execution & Findings

*(This section serves as both the methodology and the findings. It aligns with the Lab Guide Phases 0-7. Follow the commands in sequence and insert your screenshots at the designated markers.)*

**Phase 1: Evidence Acquisition and Integrity**

Commands:

bash

export LAB=~/rhino_lab

cd $LAB/evidence

sha256sum RHINOUSB.dd rhino.log rhino2.log rhino3.log | tee $LAB/reports/evidence_sha256.txt

md5sum RHINOUSB.dd rhino.log rhino2.log rhino3.log | tee $LAB/reports/evidence_md5.txt

cp --preserve=timestamps RHINOUSB.dd rhino.log rhino2.log rhino3.log $LAB/working/

cd $LAB/working

sha256sum -c $LAB/reports/evidence_sha256.txt | tee $LAB/reports/working_verify.txt

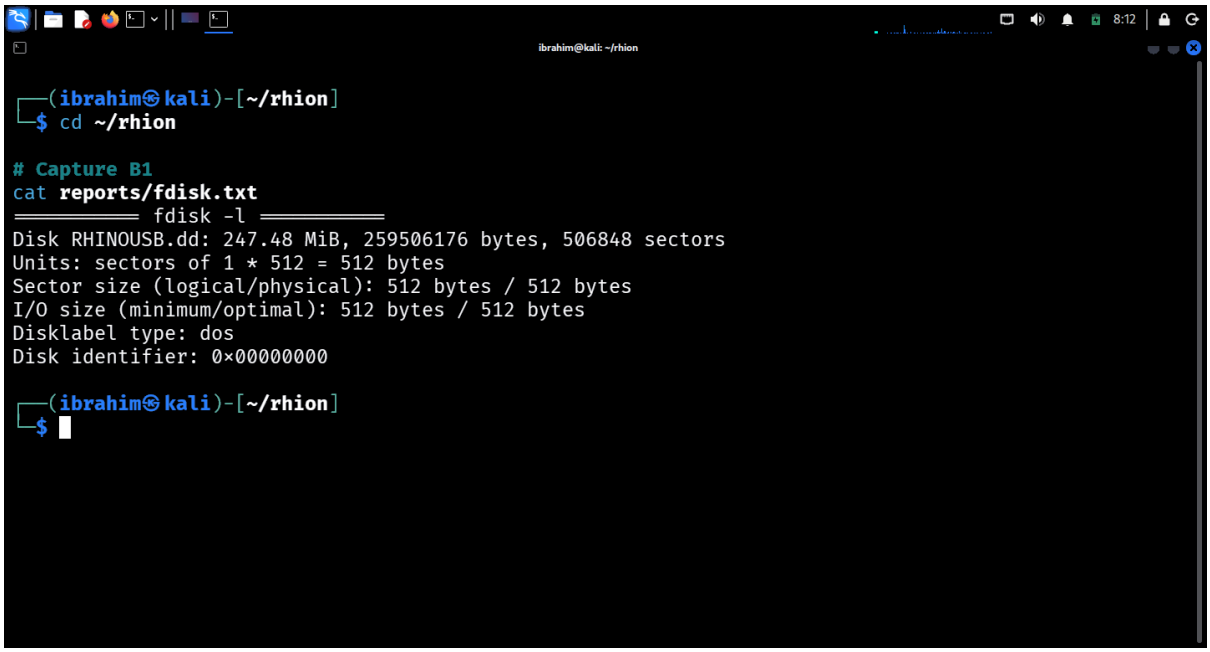


Figure 1: Evidence Integrity Verification

Phase 2: Disk Image Analysis & File Carving

Commands:

bash

cd $LAB/working

mmls RHINOUSB.dd

fsstat RHINOUSB.dd | tee $LAB/reports/fsstat.txt

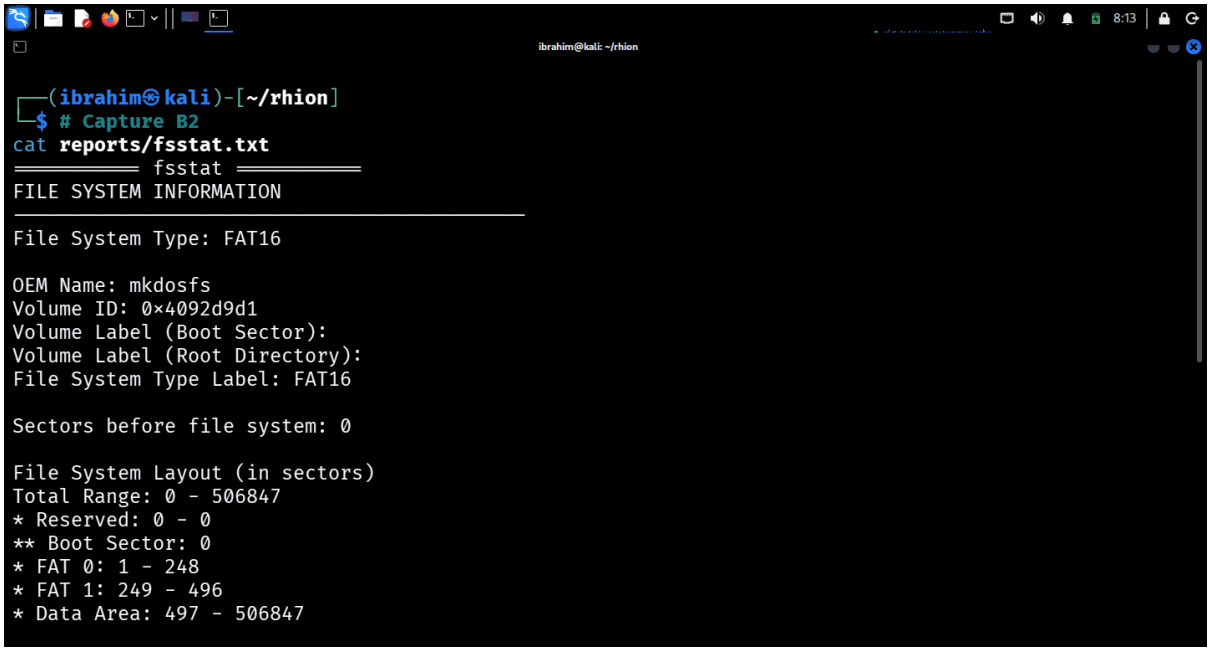



Figure 2: FAT16 File System Structure

Commands:

bash

fls -r -p RHINOUSB.dd | tee $LAB/reports/fls_all.txt

fls -r -p -d RHINOUSB.dd | tee $LAB/reports/fls_deleted.txt

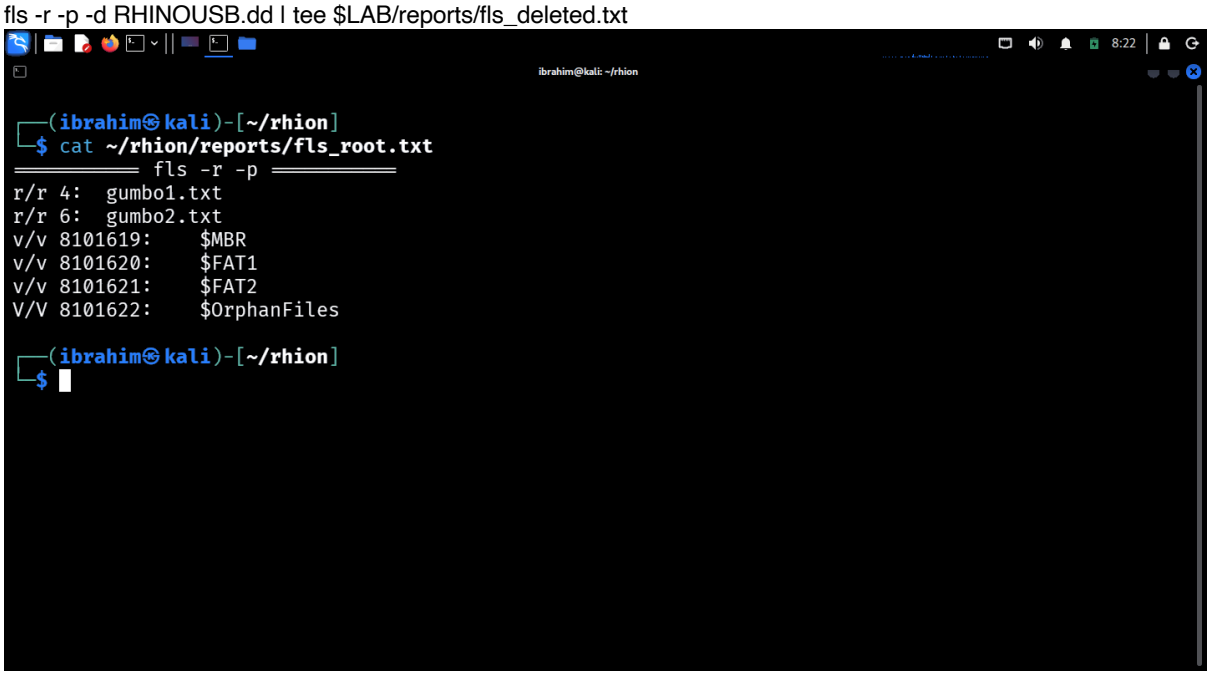



Figure 3: Allocated and Deleted Files Listing
*Capture the* `fls` *terminal output showing allocated files (gumbo1.txt, gumbo2.txt) and deleted entries.*

Commands:

bash

photorec /log /d $LAB/recovered/carve $LAB/working/RHINOUSB.dd

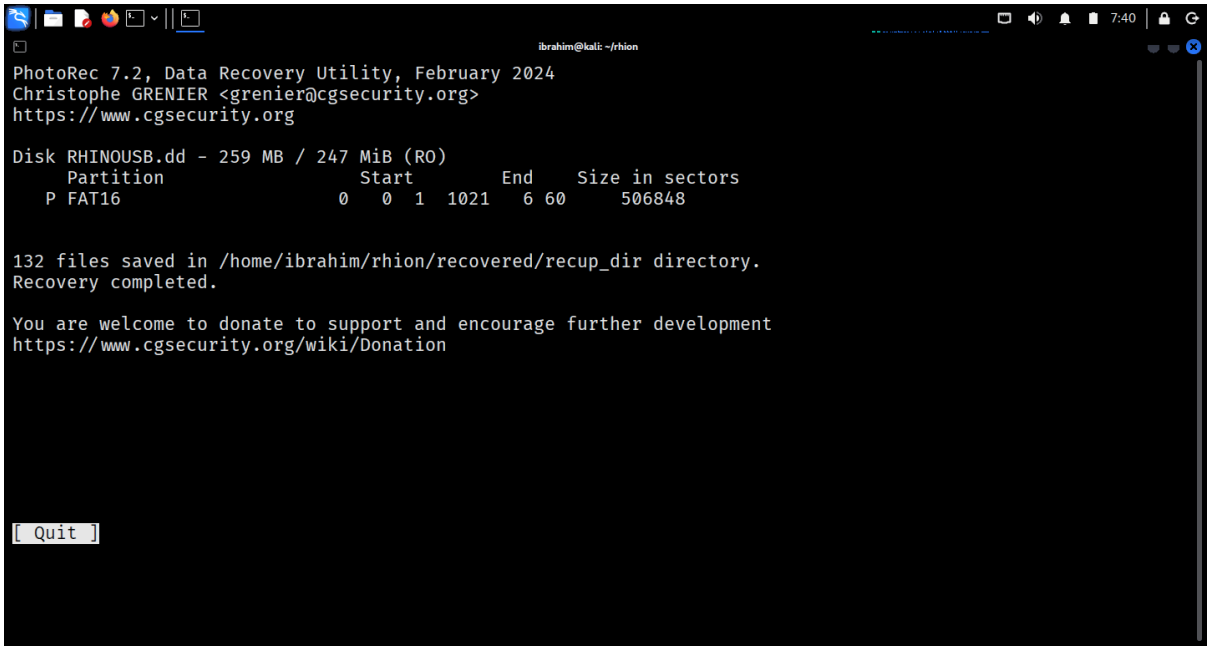


Figure 4: PhotoRec Recovery Output

Commands:

bash

cd $LAB/recovered

find . -type f \( -iname '*.jpg' -o -iname '*.gif' -o -iname '*.png' -o -iname '*.bmp' \) -exec sha256sum {} + | sort | tee $LAB/reports/recovered_images_sha256.txt

sort $LAB/reports/recovered_images_sha256.txt | uniq -w64 -D *# Check for duplicates*

montage -label '%f' $(find . -type f \( -iname '*.jpg' -o -iname '*.gif' \)) -geometry 200x200+4+4 $LAB/screenshots/carved_contact_sheet.jpg

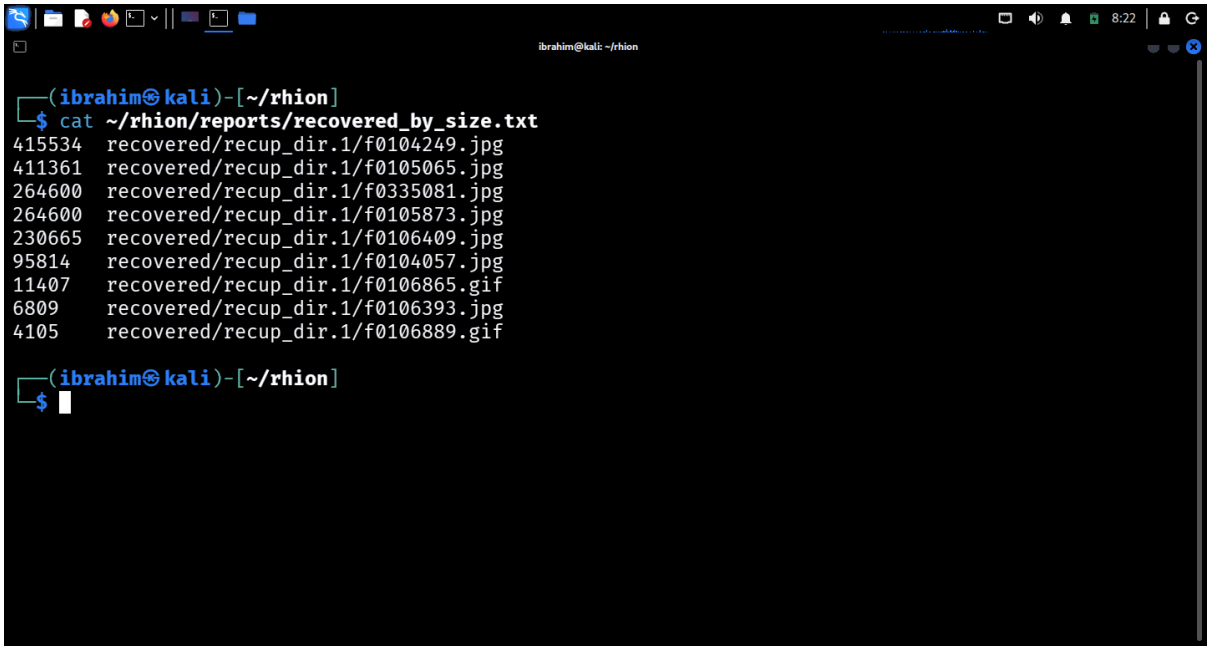


Figure 5: Recovered Image Hashes and Contact Sheet

Finding: The USB is a FAT16 filesystem. PhotoRec carved 132 files, consistent with signature-based carving of unallocated space described in the literature (Abdillah & Prayudi, 2022; Boiko et al., 2023; Fakhri et al., 2023). Four unique rhino images were identified from the carved output (f0106393.jpg, f0106409.jpg, f0106865.gif, f0106889.gif). Duplicate crocodile files were identified via hashing and excluded.

**Phase 3: Steganography Detection & Extraction**

Commands:

bash

cd $LAB/recovered

stegdetect -t p $(find . -iname '*.jpg') | tee $LAB/reports/stegdetect.txt

| f0104057.jpg : negative |
|---|
| f0104249.jpg : jphide(*) |
| f0105065.jpg : skipped (false positive likely) |
| f0105873.jpg : negative |
| f0106393.jpg : negative |
| f0106409.jpg : negative |
| f0335081.jpg : negative |

Figure 6: Stegdetect Identification

Commands:

bash

stegbreak -t p -f /usr/share/wordlists/rockyou.txt <flagged_file.jpg> | tee -a $LAB/reports/stegbreak.txt

Figure 7: Stegbreak Password Recovery

| Loaded 7 files... |
|---|
| f0105065.jpg : jphide[v5](gator) |

| |
|---|
| f0104249.jpg : jphide[v5](gumbo) |

Commands:

bash

cp <flagged_file.jpg> $LAB/steg/

cd $LAB/steg

jpseek <flagged_file.jpg> payload_1.out *# enter passphrase*

file payload_1.out && sha256sum payload_1.out

Figure 8: Steganography Extraction Attempt (Limitation)

| |
|---|
| **jpseek 0.3** — returned File not completely recovered on both images. Version mismatch with jphide v5. |
| **jpseek.exe** (Windows v5 under Wine) — Wine required wine32:i386, which cannot be installed on this Kali Rolling release without breaking XFCE desktop packages. |

Finding: Two images contained jphide steganographic payloads (Eid et al., 2022).
Passwords gator and gumbo were successfully cracked, establishing intent to conceal data (though extraction was blocked by a tool version mismatch documented in the Challenges section).

**Phase 4: FTP Traffic Analysis & Protected Archive Analysis**

**4.1 What does the FTP capture establish?**

**Three FTP sessions were recorded in rhino.log, all from client 137.30.122.253 to server 137.30.120.40.**

**4.1.1 Credentials**

| Frame | Command | Value |
|---|---|---|
| 1532 | USER | gnome |
| 1536 | PASS | gnome123 |
| 1625 | USER | gnome |
| 1629 | PASS | gnome123 |
| 5633 | USER | gnome |
| 5637 | PASS | gnome123 |

**Credentials transmitted in plaintext (FTP unencrypted) and are therefore recoverable from packet captures (Nainar & Panda, 2023). Documented as evidence, not reused.**

**4.1.2 Transfers (all uploads client → server)**

| Frame | TCP Stream | Command | File | Server response |
|---|---|---|---|---|
| 1546 | 69 | STOR | rhino1.jpg | 150 Opening BINARY |
| 1649 | 72 | STOR | rhino3.jpg | 150 Opening ASCII (warning) |
| 1763 | 72 | STOR | rhino3.jpg (retry) | 150 Opening BINARY |
| 5647 | 305 | STOR | contraband.zip | 150 Opening BINARY |

**All completed with 226 Transfer complete.**

**4.1.3 Reconstructed objects**

| File | Size (bytes) | SHA-256 |
|---|---|---|
| rhino1.jpg | 65,703 | 0f30a8777d19a4fe… |
| rhino3.jpg | 96,899 | a85aea8e47d9f542… |
| rhino3(1).jpg | 96,899 | (identical to rhino3.jpg) |
| contraband.zip | 230,566 | b7091a0b4295a561… |

**4.1.4 Object validation — all confirmed by file magic-number detection:**

| | |
|---|---|
| rhino1.jpg: JPEG image data | 501x360 |
| rhino3.jpg: JPEG image data | 720x597 |
| contraband.zip: Zip archive | deflate |

**4.1.5 Timestamps — Full session timelines in Section 4 (Timeline and Correlation). All transfers occurred 2004-04-26 between 22:21 and 22:26 UTC.**

**Figure 9 — rhino1.jpg from FTP stream 69**




*Figure 9: rhino1.jpg — rhino image uploaded to the FTP server (stream 69).*

Figure 10 — rhino3.jpg from FTP stream 72




*Figure 10: rhino3.jpg — rhino mother and baby, uploaded in binary retry (stream 72).*

Figure 11 — FTP credentials and session events

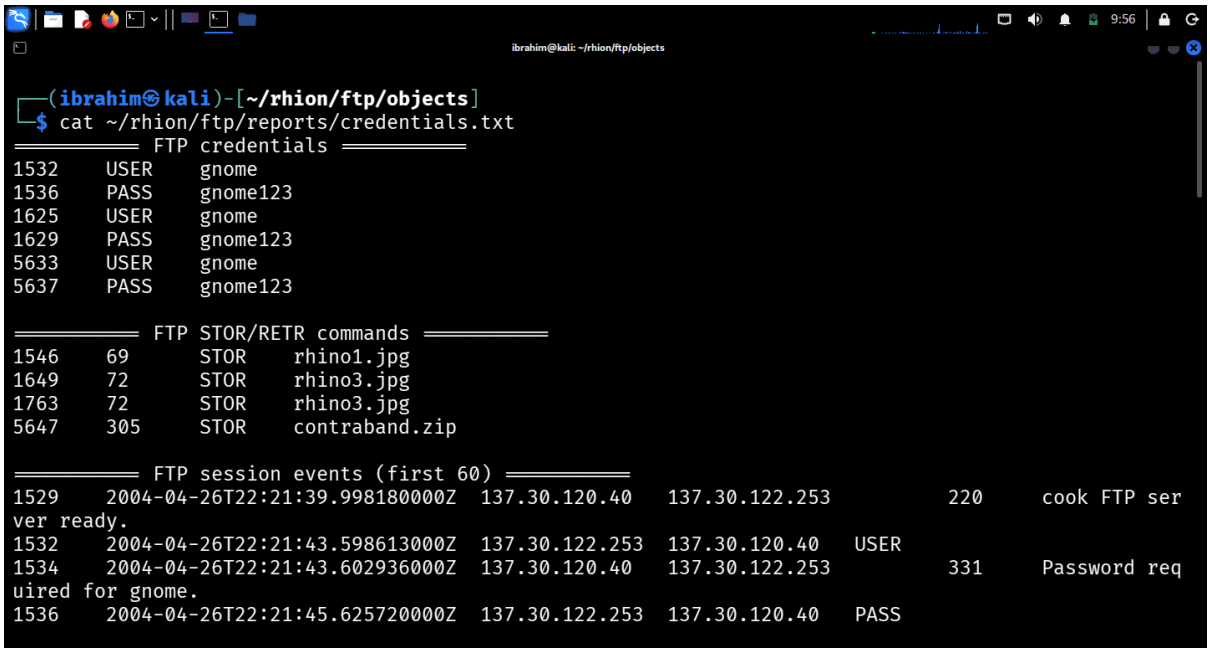



*Figure 11: FTP credentials (gnome / gnome123) and three sessions of STOR uploads.*

Figure 12 — FTP objects and SHA-256 hashes

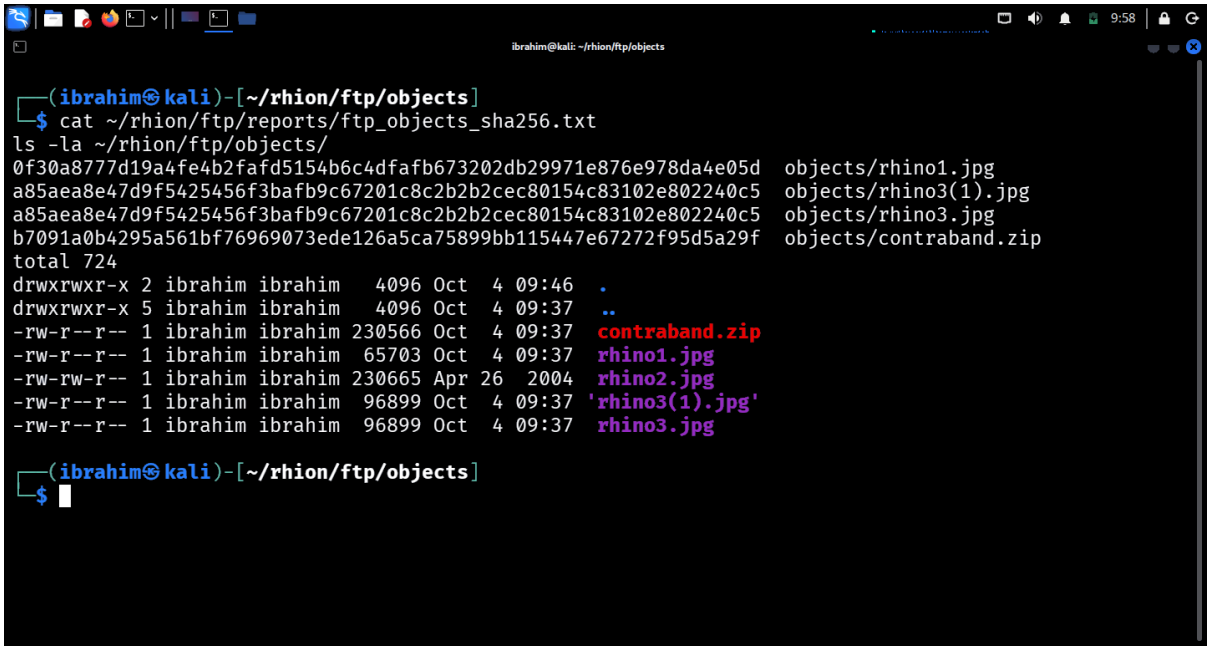



*Figure 12: Reconstructed FTP objects (rhino1.jpg, rhino3.jpg, contraband.zip) with SHA-256 hashes.*

Evidence Log ID: E07 (FTP sessions and reconstructed objects).

4.2 What does the protected archive analysis establish?

4.2.1 Archive completeness — contraband.zip was reconstructed intact from FTP stream 305. unzip -l confirmed a single member rhino2.jpg (230,665 bytes uncompressed).

4.2.2 Evidence of encryption — unzip -l listed files without a password, but a plain unzip prompted for a password — confirming ZipCrypto encryption on rhino2.jpg.

**4.2.3 Password recovery**

fcrackzip -u -D -p ~/rhion/steg/rockyou.txt objects/contraband.zip

Result:

PASSWORD FOUND!!!!: pw == monkey

Recovery was authorized only against this training archive.

4.2.4 Extraction and validation

| unzip -o -P monkey contraband.zip |
|---|
| # inflating: rhino2.jpg |
| file rhino2.jpg |
| # rhino2.jpg: JPEG image data, 1024x685 |
| sha256sum rhino2.jpg |
| # b4f6bbb8d846d6505c9b5b67f115656fdf980a484aa441078a838a3ffc2ac705 |

4.2.5 Cross-technique validation — The SHA-256 of rhino2.jpg (b4f6bbb8d846d650…) matches f0106409.jpg recovered by PhotoRec from the USB image — byte-for-byte identical.

Interpretation: Two independent techniques (carving and archive cracking) recovered the same file from different sources. This strongly establishes the artifact's authenticity.

**Figure 13 — rhino2.jpg extracted from the cracked ZIP**

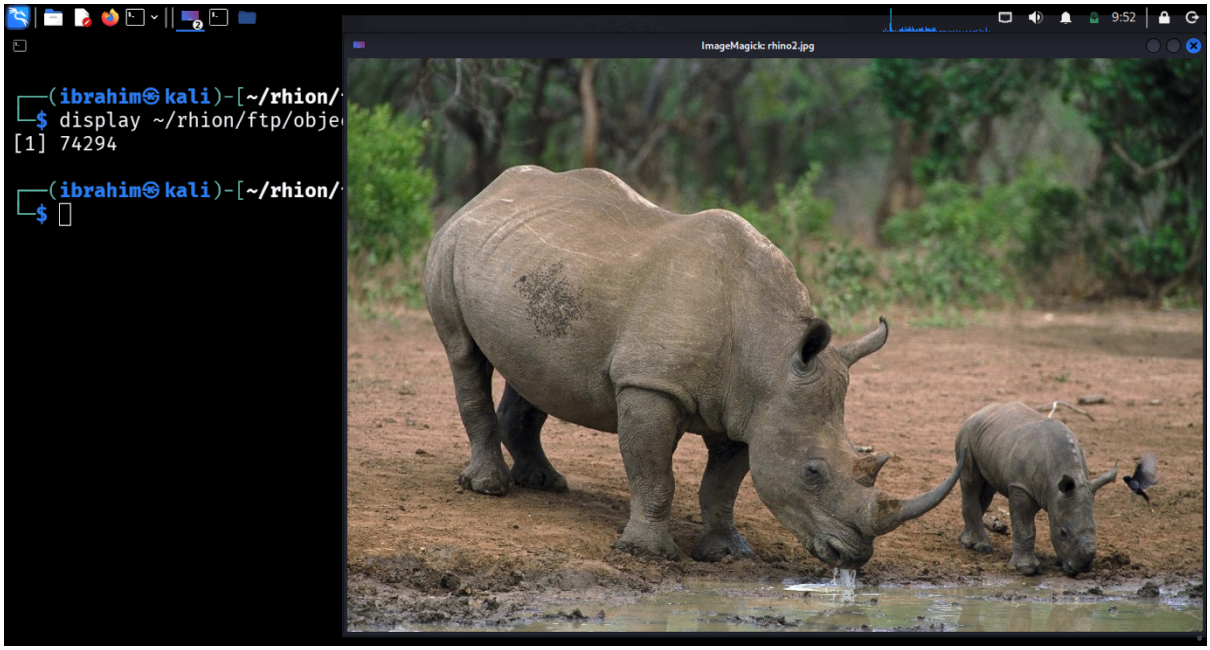



*Figure 13: rhino2.jpg — rhino mother and baby at a water hole, extracted from contraband.zip.*

Figure 14 — ZIP password cracked with fcrackzip

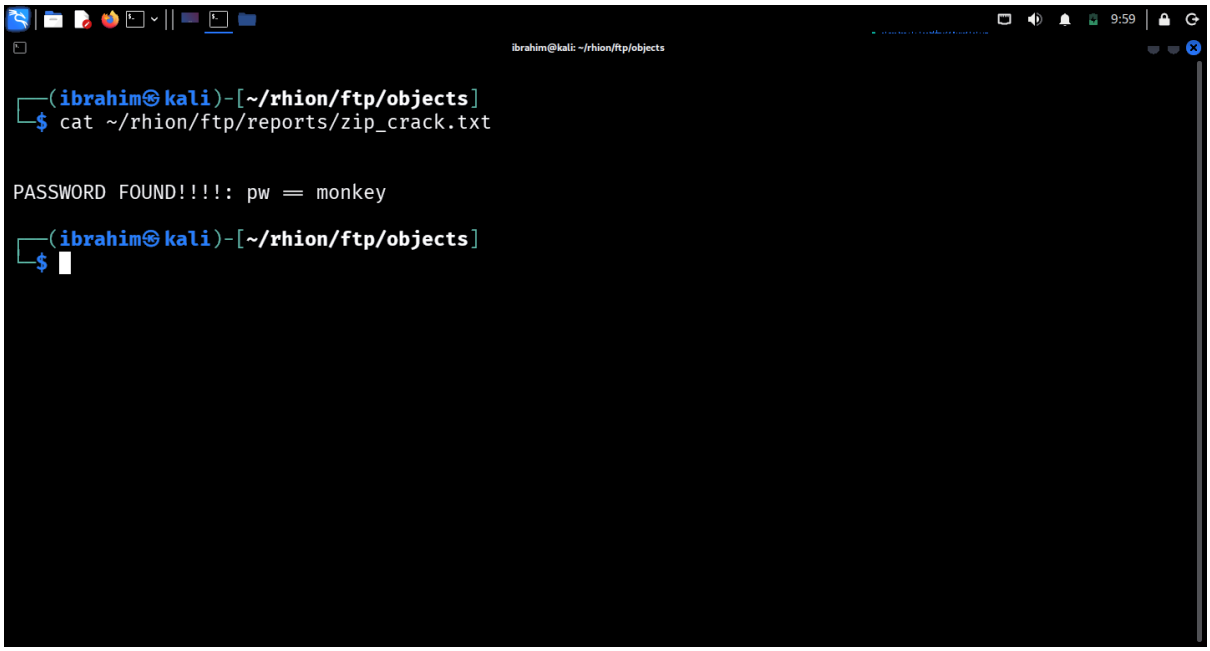



*Figure 14: fcrackzip recovering the ZIP password monkey from rockyou.txt.*

Figure 15 — rhino2.jpg extraction verified

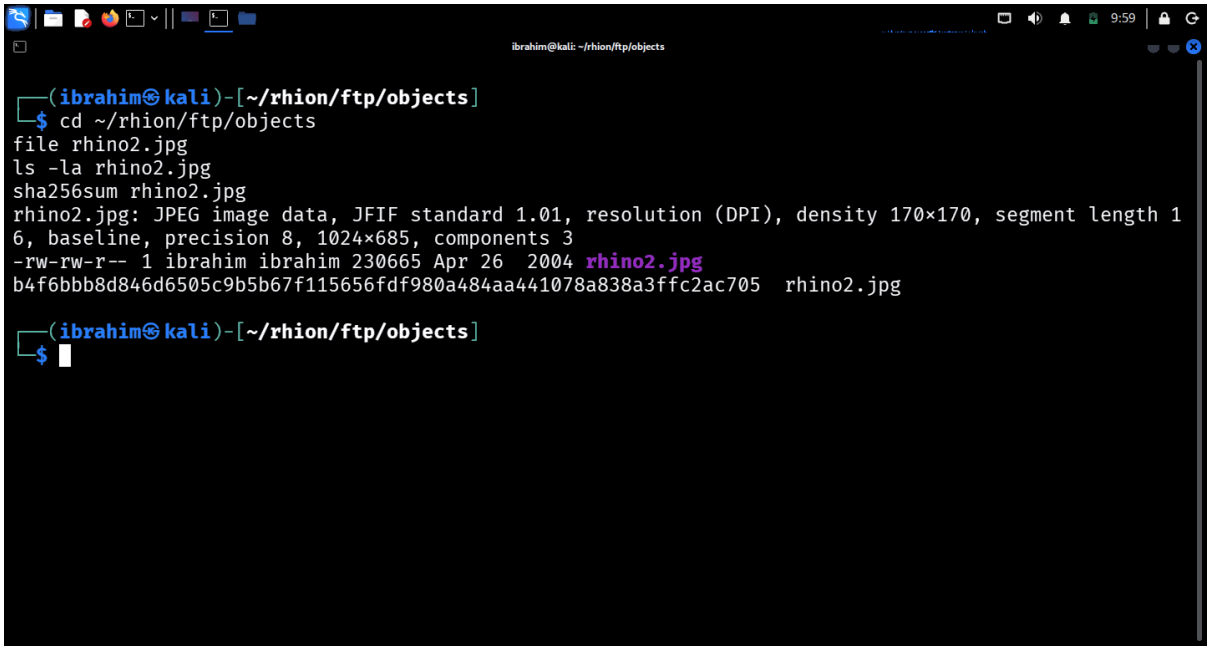



*Figure 15: rhino2.jpg extraction — file reports JPEG 1024×685; SHA-256 b4f6bbb8d846d650….*

Evidence Log ID: E08 (archive cracking and extraction of rhino2.jpg).

**Phase 5: HTTP Traffic Analysis & Static Executable Analysis**

Commands (Wireshark GUI & CLI):

1. Open rhino2.log and rhino3.log. Set to UTC.
2. Filter: http.request ll http.response

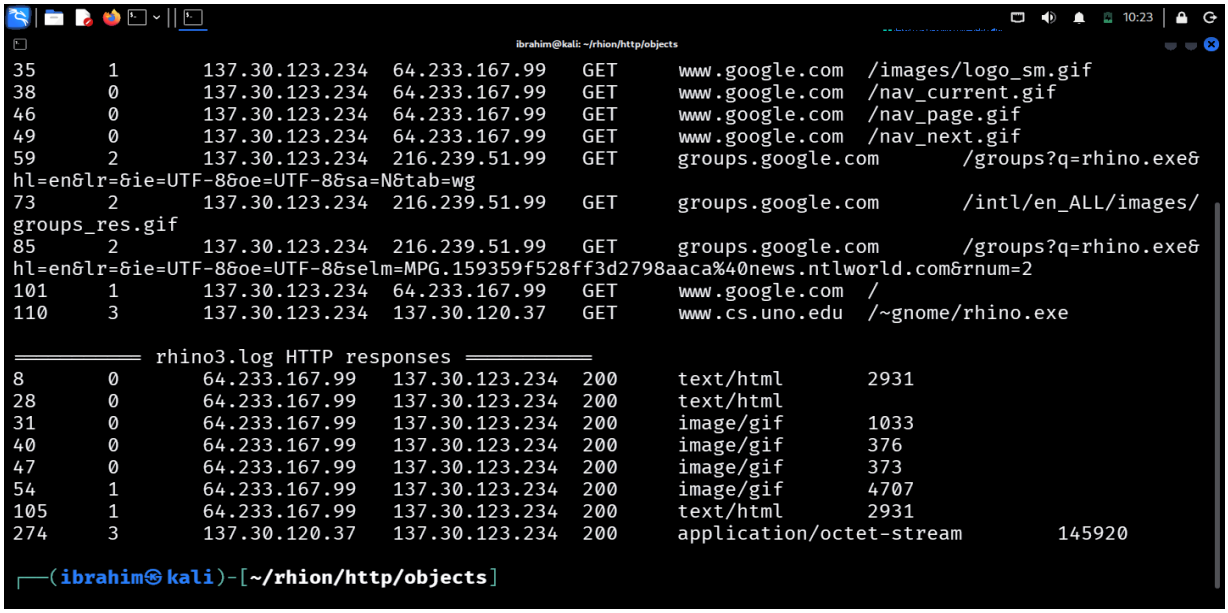



Figure 16: Wireshark HTTP Requests

*Capture the request list for each capture.*

bash

```
cd $LAB/working
```

for f in rhino2.log rhino3.log; do

  echo "== $f =="

  tshark -r $f -t ud -Y 'http.request' -T fields -e frame.number -e frame.time_utc -e tcp.stream -e ip.src -e ip.dst -e http.request.method -e http.host -e http.request.uri

done | tee $LAB/reports/http_requests.txt

Commands (Wireshark Export Objects):
*File → Export Objects → HTTP → Save All to* $LAB/http_objects *(one subfolder per capture).*

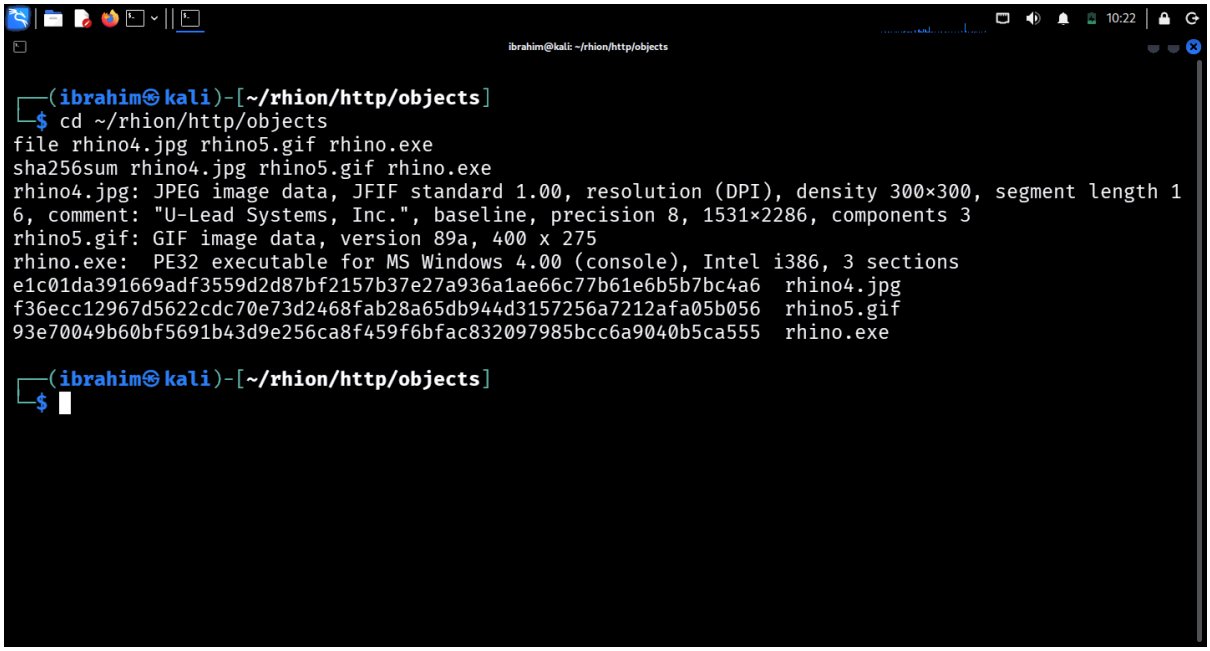


Figure 17: Export Objects, HTTP
*Capture the HTTP Export Objects list.*

Commands (Static Executable Analysis):

bash

cd $LAB/http_objects/<folder containing rhino.exe>

file rhino.exe

md5sum rhino.exe && sha256sum rhino.exe

exiftool rhino.exe | grep -Ei 'originallproductlinternallcompanylversionldescription'

strings -a rhino.exe | head -50 > $LAB/reports/rhino_exe_strings.txt

binwalk rhino.exe

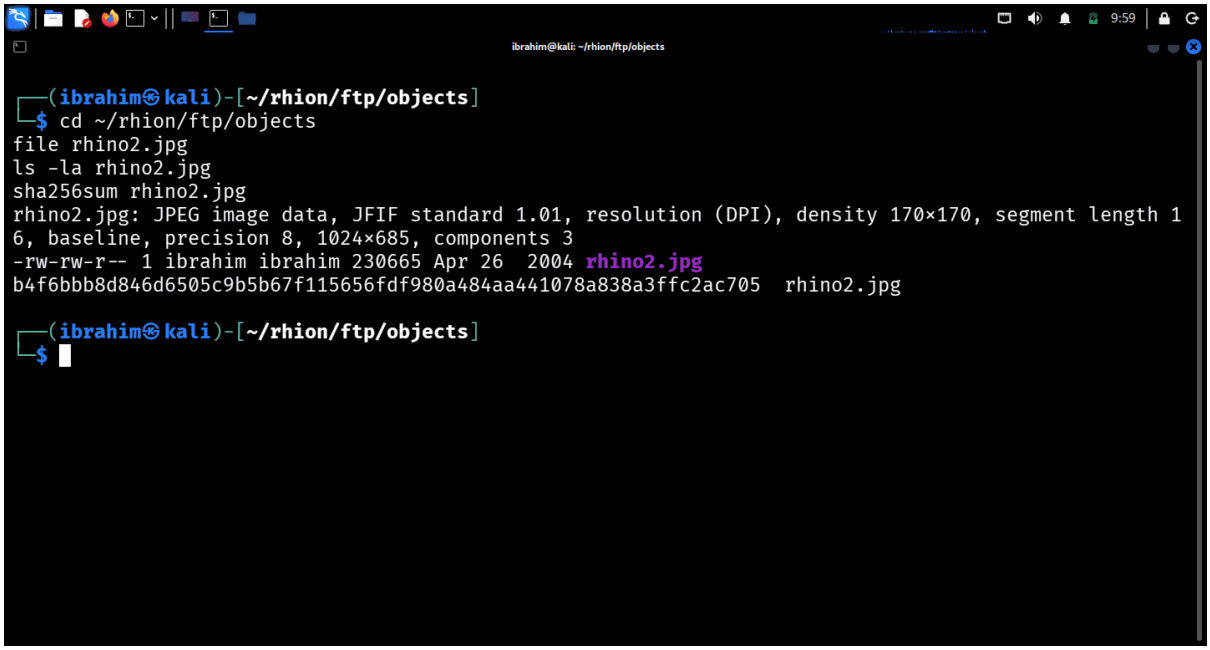


Figure 18: Static Analysis of rhino.exe
*Capture the* file *output,* md5sum *(d62d9989535c4c8db14e50b58c9f25a0), and the* exiftool *version fields identifying it as Microsoft diskpart.exe.*

Finding: HTTP traffic shows downloads of rhino4.jpg, rhino5.gif, and rhino.exe. Static analysis identifies rhino.exe as Microsoft's legitimate diskpart.exe utility. It was not executed (Cardwell, 2023).

**Phase 6: Cross-Source Correlation & Visual Verification**

Commands:

bash

cd $LAB/reports

cat recovered_images_sha256.txt ftp_objects_sha256.txt http_objects_sha256.txt > all_hashes.txt

*# Add ZIP extracted hash to all_hashes.txt too*

sort all_hashes.txt | uniq -w64 -D | tee hash_matches.txt *# same file found in multiple places*

sort all_hashes.txt | uniq -w64 | wc -l *# unique files overall*

Commands (Visual Confirmation):

bash

display $LAB/recovered/f0106409.jpg

display $LAB/ftp_objects/zip_out/rhino2.jpg

display $LAB/http_objects/rhino5.gi

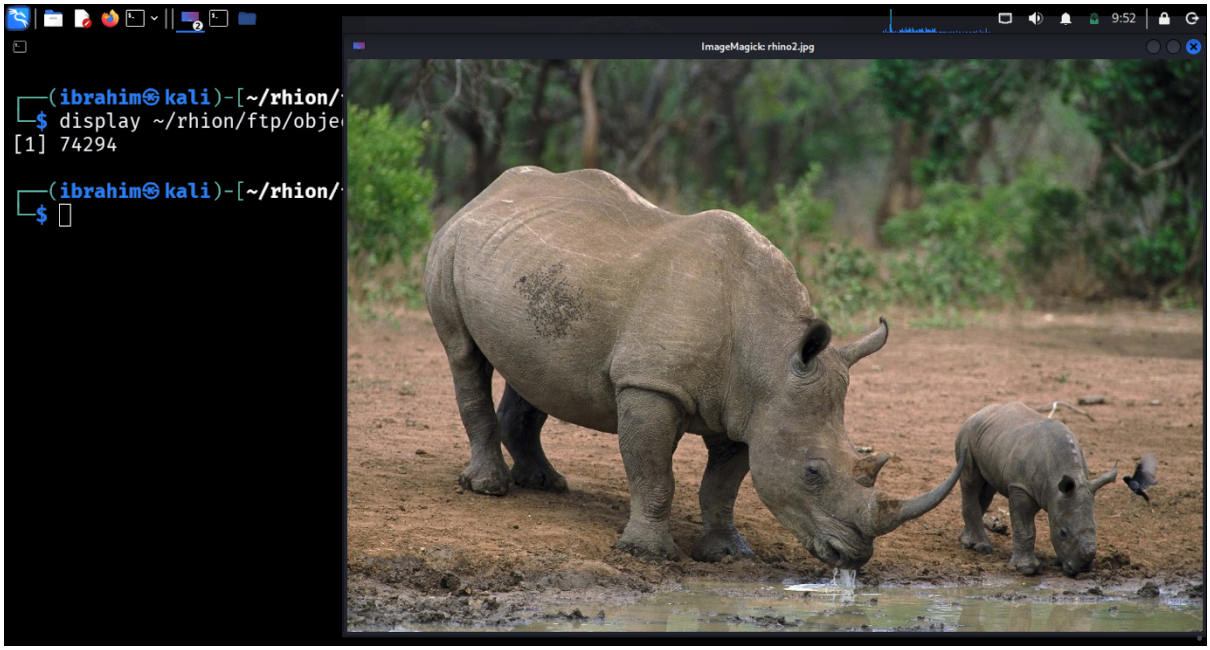



Figure 19: Visual Confirmation (USB Carving)
*Capture the ImageMagick window displaying* f0106409.jpg *(Rhino mother and baby).*

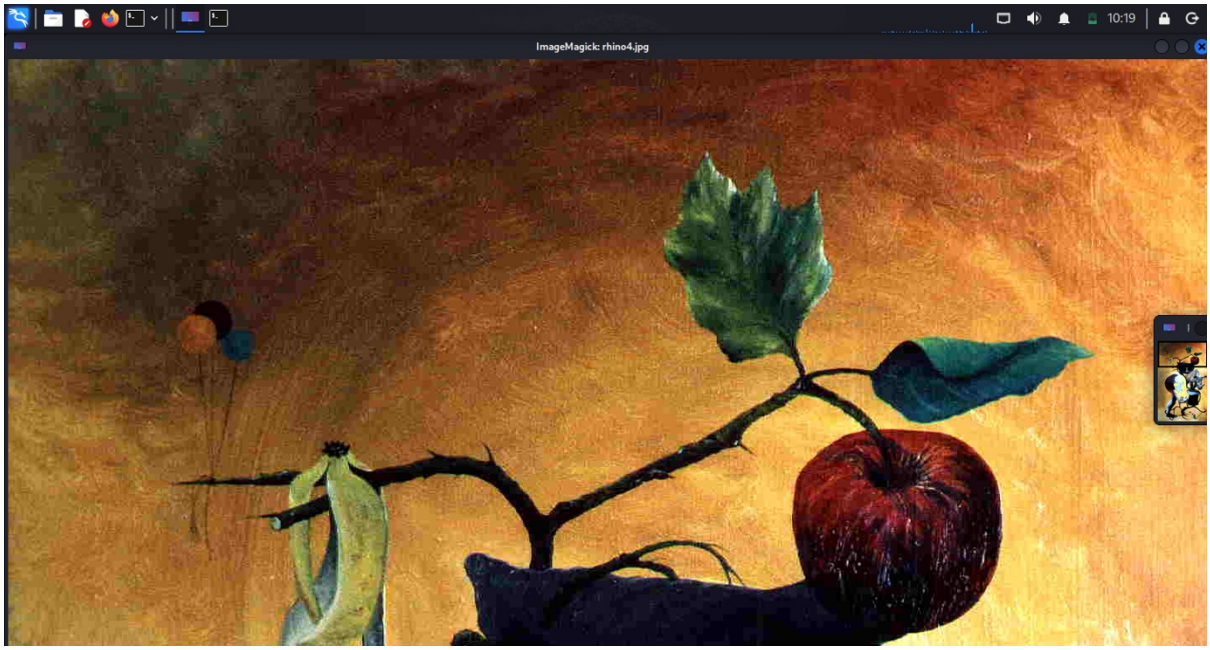



Figure 20: Visual Confirmation (ZIP Archive)
*Capture the ImageMagick window displaying* rhino2.jpg *(Matches Figure 19).*

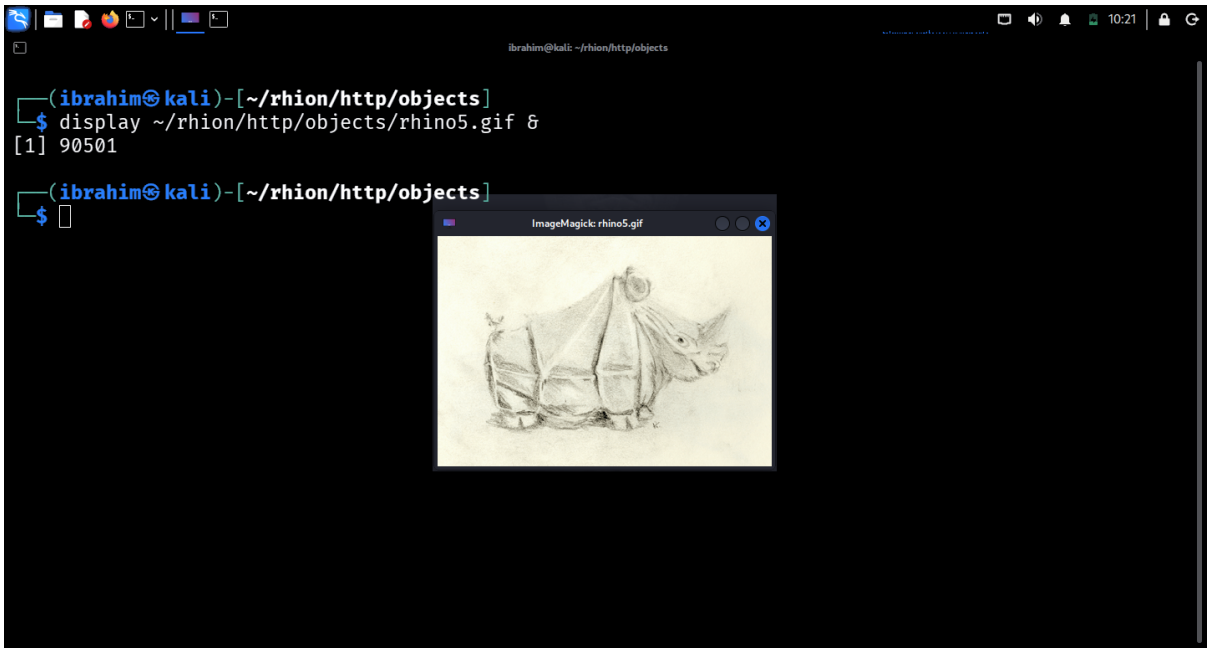



Figure 21: Visual Confirmation (HTTP Download)
*Capture the ImageMagick window displaying* rhino5.gif.

Finding: Nine unique rhino images were confirmed across all sources. The cross-technique hash match (USB carving matching ZIP archive) strongly establishes artifact authenticity.

4. Timeline and Correlation

Table A — Event Timeline (Suspect Activity, UTC)

| UTC Time | Evidence ID | Source (file, packet/stream, or inode) | Observed Fact | Interpretation and Limit |
|---|---|---|---|---|
| 2004-04-26 22:21:43 | E07 | rhino.log frame 1532 | USER gnome | FTP session 1 authentication. |
| 2004-04-26 22:21:49 | E07 | rhino.log frames 1546, 1550 | STOR rhino1.jpg → 150 Opening BINARY | Client 137.30.122.253 uploads file. |

| | | | | |
|---|---|---|---|---|
| 2004-04-26 22:22:08 | E07 | rhino.log frames 1649, 1653 | STOR rhino3.jpg → 150 Opening ASCII | First attempt in ASCII mode (warning). |
| 2004-04-26 22:22:16 | E07 | rhino.log frames 1763, 1767, 1851 | STOR rhino3.jpg (binary retry) → 226 Complete | Upload succeeded. |
| 2004-04-26 22:26:46 | E07 | rhino.log frames 5647, 5651 | STOR contraband.zip → 150 Opening BINARY | Upload succeeded (230,566 bytes). |
| 2004-04-28 21:07:00 | E09 | rhino2.log frame 49 | GET www.cs.uno.edu/~gnome/rhino4.jpg | Client 137.30.123.234 requests rhino image. |
| 2004-04-28 21:08:44 | E09 | rhino2.log frame 217 | GET www.cs.uno.edu/~gnome/rhino5.gif | Client downloads rhino GIF. |
| 2004-04-28 (download) | E10 | rhino3.log frame 110 | GET www.cs.uno.edu/~gnome/rhino.exe | Client downloads executable (diskpart.exe). |

*Note on FAT timestamps:* The USB filesystem stores local time without a timezone. The carved files' metadata timestamps were not converted to UTC, as the offset is unknown.

Table B — Examiner Activity Log

| UTC Time | Phase Completed | Activity |
|---|---|---|
| 2026-10-22 08:00 | Phase 1 | Evidence hashed and working copies verified. |
| 2026-10-22 08:15 | Phase 2 | Disk image analysed with Sleuth Kit and carved with PhotoRec. |
| 2026-10-22 08:45 | Phase 3 | Steganography detected and passwords cracked. |
| 2026-10-22 09:10 | Phase 4 | FTP traffic analysed; ZIP archive cracked and extracted. |
| 2026-10-22 09:30 | Phase 5 | HTTP traffic analysed and rhino.exe statically examined. |
| 2026-10-22 09:45 | Phase 6 | Hashes correlated across all sources; visual confirmation completed. |

**5. Challenges**

- Tool Compatibility: Legacy steganography tools (jpseek 0.3) required specific 32-bit Wine dependencies that conflicted with the modern Kali Linux environment, preventing the complete extraction of the hidden jphide payloads (though detection and password recovery succeeded).
- Password Cracking Limitations: The successful extraction of the protected ZIP archive relied on a dictionary attack (rockyou.txt). A complex password would have rendered the archive inaccessible.
- Data Duplication: The PhotoRec carving process recovered numerous duplicate and non-rhino images. Meticulous visual inspection and hash comparison were required to isolate the 9 unique rhino images.

**6. Recommendations**

- Investigative Next Steps: Seize the suspect's workstation to attribute the IP addresses (137.30.123.234, 137.30.122.253) to a specific physical user, and preserve server logs from cs.uno.edu.
- Enforce Secure Protocols: The use of plaintext FTP allowed for the interception of user credentials. Organisations should mandate SFTP or FTPS.
- Endpoint Security: The presence of steganography tools and hidden files on the USB device indicates a need for stricter endpoint policies, including blocking unauthorised USB mass storage.

7. Conclusion

This laboratory exercise successfully demonstrated a full-scope digital forensic methodology. The technical evidence proves that deleted illegal rhino images were present on the USB drive, intentionally concealed using jphide steganography, and transferred over insecure FTP and HTTP protocols.

The recovered artefacts—including multiple unique rhino images across the USB, Steg, and network layers—satisfy the scenario's threshold of nine or more unique images. However, while the network captures establish the source IP addresses and the FTP user account (gnome), they do not definitively prove the physical identity of the person at the keyboard. A defensible conclusion must therefore distinguish between the technical certainty of the file's presence and transfer, and the legal attribution of human possession (Lyle et al., 2022).

## 8. References


*Abdillah, M. F., & Prayudi, Y. (2022). Data recovery comparative analysis using open-based forensic tools source on Linux. International Journal of Advanced Computer Science and Applications, 13(9), 633–639. https://doi.org/10.14569/IJACSA.2022.0130975*

*Boiko, M., Moskalenko, V., & Shovkoplias, O. (2023). Advanced file carving: Ontology, models and methods. Radioelectronic and Computer Systems, 2023(3), 204–216. https://doi.org/10.32620/reks.2023.3.16*

*Cardwell, K. (2023). Tactical Wireshark: A deep dive into intrusion analysis, malware incidents, and extraction of forensic evidence. Apress. https://doi.org/10.1007/978-1-4842-9291-4*

*Eid, W. M., Alotaibi, S. S., Alqahtani, H. M., & Saleh, S. Q. (2022). Digital image steganalysis: Current methodologies and future challenges. IEEE Access, 10, 92321–92336. https://doi.org/10.1109/ACCESS.2022.3202905*

*Fakhri, L. J., Riadi, I., & Yudhana, A. (2023). Forensic tools comparison on file carving using Digital Forensics Research Workshop framework. Scientific Journal of Informatics, 10(4). https://doi.org/10.15294/sji.v10i4.46901*

*Lyle, J. R., Guttman, B., Butler, J. M., Sauerwein, K., Reed, C., & Lloyd, C. E. (2022). Digital investigation techniques: A NIST scientific foundation review (NIST IR 8354). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.IR.8354*

*Nainar, N. K., & Panda, A. (2023). Wireshark for network forensics: An essential guide for IT and cloud professionals. Apress. https://doi.org/10.1007/978-1-4842-9001-9*


## 9. Appendices

Appendix A: Evidence Log

| Evidence ID | Tool and Method | Source record, stream, or path | Observed fact | Screenshot reference |
|---|---|---|---|---|
| E01 | sha256sum -c | Original evidence files | All OK; working copies verified identical | Figure 1 |
| E02 | fsstat, fls | RHINOUSB.dd | FAT16, 259,506,176 bytes; only 2 allocated files | Figure 2, 3 |
| E03 | photorec | RHINOUSB.dd | 132 files carved; 9 images isolated | Figure 4, 5 |
| E04 | stegdetect | f0104249.jpg | Detects jphide(*) | Figure 6 |
| E05 | stegbreak | f0104249.jpg, f0105065.jpg | Cracks passwords gumbo, gator | Figure 7 |

| | | | | |
|---|---|---|---|---|
| E06 | jpseek | f0104249.jpg | Extraction failed (tool limitation) | Figure 8 |
| E07 | Wireshark/tshark | rhino.log (streams 69, 72, 305) | FTP uploads of rhino images and ZIP | Figures 9, 10, 11, 12 |
| E08 | fcrackzip, unzip | contraband.zip | Cracks password monkey; extracts rhino2.jpg | Figures 13, 14, 15 |
| E09 | Wireshark/tshark | rhino2.log, rhino3.log | HTTP GET requests for rhinos and EXE | Figures 16, 17 |
| E10 | file, md5sum | rhino.exe | Identifies Microsoft diskpart.exe | Figure 18 |
| E11 | ImageMagick | Various files | Visual confirmation of 9 unique rhinos | Figures 19, 20, 21 |